Visualization Draft

Phoebe Dupa, Rebecca Martinez, and Shucan Zhao

1. Set up

Packages

```
# general use (data wrangling and plotting)
library(tidyverse)

# clean column names
library(janitor)

# file organization
library(here)
```

Data

```
# weather data from NOAA weather station
weather <- read_csv(here("data", "NOAA-weather-data.csv"))

# water quality survey metadata
metadata <- read_csv(here("data", "NCOS_YSI_Water_Quality_Monitoring_0.csv"))

# water quality parameter measurements
water_parameters <- read_csv(here("data", "YSI_Data_Begin_1.csv"))
```

Cleaning weather

```
# creating new object from weather
weather_clean <- weather |>
  # cleaning column names
  clean_names() |>
  # making sure date is in POSIXct format
  mutate(date = mdy(date)) |>
  # selecting columns of interest
  select(date, prcp, tmax, tmin) |>
  # extracting month and year from date column
  mutate(month = month(date),
         year = year(date)) |>
  # assigning water year
  mutate(wy = case_when(
    # when month is October or later, assign value of year + 1
    month >= 10 ~ year + 1,
    # when month is September or earlier, assign value of existing year
    .default = year))
```

Cleaning metadata

```
# creating new object from metadata
metadata_clean <- metadata |>
  # cleaning column  names
  clean_names() |>
  # excluding unnecessary columns
  select(!c(object_id, creator, editor)) |>
  # making sure monitoring_date is in POSIXct format
  mutate(monitoring_date = mdy_hms(monitoring_date)) |>
  # renaming monitoring datetime column to reflect values
  rename(monitoring_datetime = monitoring_date) |>
  # creating new columns for month and year of observations
  mutate(month = month(monitoring_datetime),
         year = year(monitoring_datetime)) |>
  # assigning water year
  mutate(wy = case_when(
    # when month is October or later, assign value of year + 1
    month >= 10 ~ year + 1,
    # when month is September or earlier, assign value of existing year
    .default = year)) |>
  # create new column of full site name
```

```
  mutate(site_full = case_when(
    site_name == "east_channel" ~ "East Channel",
    site_name == "phelps_bridge" ~ "Phelps Bridge",
    site_name == "venoco_bridge" ~ "Venoco Bridge",
    site_name == "other" ~ "Other"
  ))
```

Cleaning water parameters

```
# creating new object from water_parameters
water_parameters_clean <- water_parameters |>
  # cleaning object names
  clean_names() |>
  # excluding unnecessary columns
  select(!c(creator, editor)) |>
  # renaming columns
  rename(elevation_code = depth_code) |>
  # replacing incorrect elevation code from 2021-07-16
  mutate(elevation_code = case_when(
    elevation_code == 10 ~ 1,
    .default = elevation_code
  )) |>
  # set factors
  mutate(elevation_code = as_factor(elevation_code),
         elevation_code = fct_relevel(elevation_code,
                                      "0", "1", "2", "3", "4", "5", "6"))
```

Joining data frames

```
# join metadata and water parameters
water_quality <- full_join(
  # left
  x = metadata_clean,
  # right
  y = water_parameters_clean,
  # shared column
  by = c("global_id" = "parent_global_id")) |>
```

```
  # stop and look at colnames(water_quality) before continuing
  # selecting relevant columns
  select(
    # information about observation
    global_id, monitoring_datetime, time_of_wq_measurement,
    monitor_name, weather, site_name, other_site_name, site_full, x, y,

    # water information
    water_surface_elevation, flow, notes,

    # measurement information
    elevation_code, do_mg_l, do_percent_sat, conductivity_u_s_cm,
    salinity_ppt, temperature_c) |>
  # extract date from monitoring datetime
  mutate(date = date(monitoring_datetime)) |>
  # move date column to appear to the right of monitoring_datetime
  relocate(date, .after = monitoring_datetime) |>
  # exclude monitoring_datetime
  select(!monitoring_datetime)
```

Filtering joined data

```
# creating filtered object for plotting and summaries
do_filtered <- water_quality |>
  # keep only the three main NCOS sites
  filter(site_full %in% c("East Channel", "Phelps Bridge", "Venoco Bridge"))
   ↪  |>
  # remove missing DO values
  filter(!is.na(do_mg_l)) |>
  # remove missing elevation codes
  filter(!is.na(elevation_code)) |>
  # drop the unused "Other" level from site_full so it doesn't appear in
   ↪  plots
  mutate(site_full = fct_drop(as_factor(site_full))) |>
  # order sites for consistent plotting (upstream -> downstream-ish)
  mutate(site_full = fct_relevel(site_full,
                                 "Phelps Bridge", "Venoco Bridge", "East
                                  ↪  Channel"))
```

Summarizing dissolved oxygen

DO summary across sites

```
# summary of DO by site (for Figure 1: median points across sites)
do_site_summary <- do_filtered |>
  # group by site
  group_by(site_full) |>
  # calculate summary statistics
  summarize(
    mean_do   = mean(do_mg_l, na.rm = TRUE),
    median_do = median(do_mg_l, na.rm = TRUE),
    sd_do     = sd(do_mg_l, na.rm = TRUE),
    n         = n(),
    .groups   = "drop"
  )
```

DO summary by elevation code and site

```
# summary of DO by site and elevation code
# (for Figure 3: median points/lines across elevation codes)
do_elevation_summary <- do_filtered |>
  # group by site and elevation code
  group_by(site_full, elevation_code) |>
  # calculate summary statistics
  summarize(
    mean_do   = mean(do_mg_l, na.rm = TRUE),
    median_do = median(do_mg_l, na.rm = TRUE),
    n         = n(),
    .groups   = "drop"
  )
```

Subset for Figure 2 (DO vs. temperature)

```
# remove missing temperature values for the DO vs. temperature plot
do_temp <- do_filtered |>
  filter(!is.na(temperature_c))
```

Figure 1

Dissolved oxygen across sites

- Start with `do_filtered`
- Use `ggplot()` to create the plot
- Use `aes(x = site_full, y = do_mg_l)` to map site and DO
- Use `geom_jitter()` to show individual DO observations
- Use `stat_summary(fun = median, geom = "point")` to show the median DO for each site
- Use `labs()` to label the axes and title
- Use `theme_bw()` for a clean theme

Figure 2

Dissolved oxygen and water temperature

- Start with `do_filtered`
- Use `filter(!is.na(temperature_c))` to remove missing temperature values
- Use `ggplot()` to create the plot
- Use `aes(x = temperature_c, y = do_mg_l, color = elevation_code)` to map temperature, DO, and elevation code
- Use `geom_point()` to show individual observations
- Use `facet_wrap(~ site_full)` to compare sites
- Use `labs()` to label the axes, title, and legend
- Use `theme_bw()` for a clean theme

Figure 3

Dissolved oxygen elevation profile

- Start with `do_filtered`
- Use `filter(!is.na(elevation_code))` to remove missing elevation values
- Use `ggplot()` to create the plot
- Use `aes(x = elevation_code, y = do_mg_l, group = 1)` to map elevation code and DO
- Use `geom_point()` or `geom_jitter()` to show individual DO observations
- Use `stat_summary(fun = median, geom = "point")` to show median DO at each elevation code
- Use `stat_summary(fun = median, geom = "line")` to connect median DO values across elevation codes

- Use `facet_wrap(~ site_full)` to compare sites
- Use `coord_flip()` to make the plot look more like an elevation profile
- Use `labs()` to label the axes and title
- Use `theme_bw()` for a clean theme